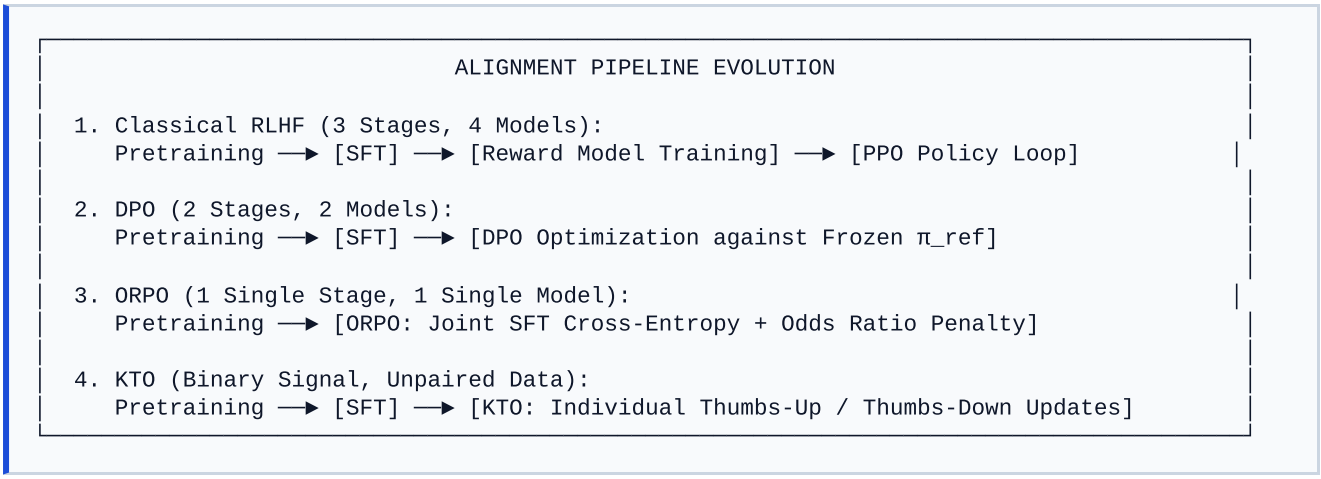
TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 10 (Part 3): Fine-Tuning, PEFT, Alignment & Distributed Training

Module 10 Part 3 completes the study of Large Language Model fine-tuning, alignment, distributed scaling, and weight-space post-processing: next-generation alignment algorithms (ORPO & KTO), preference data curation, evaluation and degradation diagnosis, the production tooling ecosystem, distributed training frameworks (DeepSpeed ZeRO & FSDP), and advanced model merging algorithms (SLERP, TIES, DARE).

10.15 Modern Alignment Simplifications: ORPO and KTO

While Direct Preference Optimization (DPO) eliminated the need for a separate Reward Model and the instability of PPO, it still requires a **two-stage pipeline**: first Supervised Fine-Tuning (SFT) to establish an initial instruction-following policy π^{SFT}, followed by a distinct preference optimization stage that holds a frozen reference model π_{ref} in memory. Modern alternatives simplify this process further.



1. Monolithic Alignment: Odds Ratio Preference Optimization (ORPO)

Introduced by Hong et al. (2024), **ORPO** eliminates the separate SFT phase entirely. It demonstrates that standard cross-entropy loss applied during SFT inadvertently increases the probability mass of *both* desired and undesired responses (as long as they share stylistic or grammatical patterns).

ORPO combines a standard Supervised Fine-Tuning cross-entropy loss with an **Odds Ratio (OR)** penalty term directly into a single objective function:

$$L_{\text{ORPO}} = L_{\text{SFT}} + \lambda_{\text{OR}} \cdot L_{\text{OR}}$$

Mathematical Formulation

The SFT loss is the standard negative log-likelihood on the chosen completion y_w:

$$L_{\text{SFT}} = -\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log P_\theta(y_{w,t} \mid x, y_{w,<t})$$

The odds of generating a sequence y given input x is defined as the probability of generating y divided by the probability of *not* generating y:

$$\text{odds}_\theta(y \mid x) = \frac{P_\theta(y \mid x)}{1 - P_\theta(y \mid x)}$$

The Odds Ratio between the winning sequence y_w and losing sequence y_l is:

$$\text{OR}_\theta(y_w, y_l) = \frac{\text{odds}_\theta(y_w \mid x)}{\text{odds}_\theta(y_l \mid x)}$$

The relative preference penalty optimizes the log odds ratio via a sigmoid function:

$$L_{\text{OR}} = -\log \sigma\left(\log \text{OR}_\theta(y_w, y_l)\right) = -\log \sigma\left(\log\left(\frac{P_\theta(y_w \mid x)}{1 - P_\theta(y_w \mid x)}\right) - \log\left(\frac{P_\theta(y_l \mid x)}{1 - P_\theta(y_l \mid x)}\right)\right)$$

- **Primary Benefit:** Requires **zero reference model (π_{ref})** in GPU memory, saves 50% VRAM compared to DPO, and merges alignment directly into the primary SFT training run.

2. Kahneman-Tversky Optimization (KTO)

Introduced by Ethayarajh et al. (Contextual AI, 2024), **KTO** departs from utility theory and operationalizes **Prospect Theory** (for which Daniel Kahneman won the Nobel Prize in Economics).

- **The Structural Flaw of Preference Pairs ($y_w \succ y_l$):** In real-world data collection, obtaining pairwise preferences is difficult and expensive. Collecting user interaction data (e.g., product clicks, upvotes, downvotes, acceptance of code suggestions) produces **unpaired binary labels**: $x \to y$ was either *good (desirable)* or *bad (undesirable)*.
- **Loss Function Formulation:** KTO optimizes utility directly on single examples:

  $$L_{\text{KTO}}(\theta, \pi_{\text{ref}}) = \mathbb{E}_{x,y}\left[w(y)\left(1 - \sigma\left(\lambda_y\left(r_\theta(x, y) - z_{\text{ref}}\right)\right)\right)\right]$$

  where:

  $$r_\theta(x, y) = \beta \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

  and z_{ref} is a moving-average reference point (the "status quo" threshold).

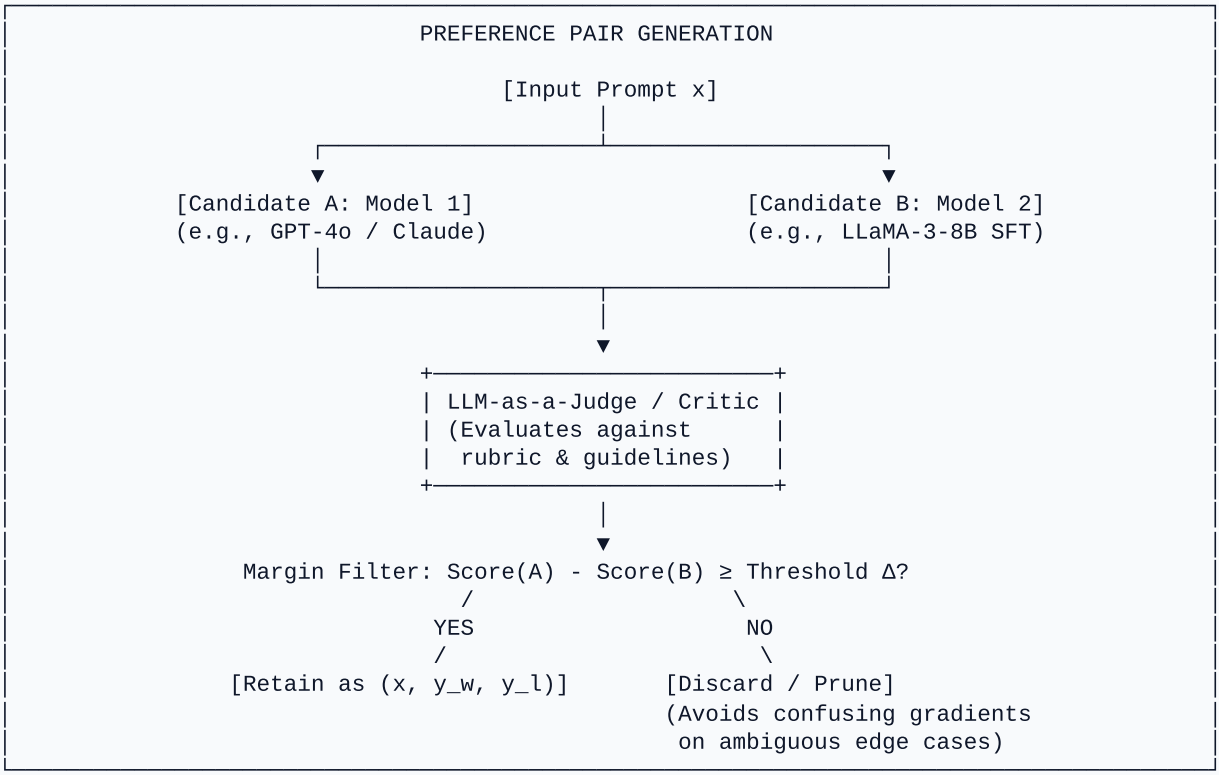
- **Loss Aversion ($w(y)$ Weighting):** Reflecting human psychology—where the pain of a loss is felt roughly twice as intensely as the pleasure of an equivalent gain—KTO sets higher weight penalties on undesirable outputs ($w(y_{dispreferred}) > w(y_{preferred})$).

3. Selection Matrix: When to Use Which Method

Paradigm	Data Requirement	Memory Footprint	Number of Stages	Ideal Production Use Case
DPO	Pairwise: (x, y_w, y_l)	2 models (Policy π_θ + Reference π_{ref})	2 stages (SFT → DPO)	When high-quality paired comparative datasets exist and stable convergence is required.
ORPO	Pairwise: (x, y_w, y_l)	**1 model** (Zero reference model needed)	**1 single stage** (Direct from base)	Resource-constrained fine-tuning; avoids running separate SFT and alignment passes.
KTO	**Unpaired binary:** $(x, y,$ is_desirable$)$	2 models (Policy π_θ + Reference π_{ref})	2 stages (SFT → KTO)	When data comes from real-world telemetry (e.g., thumbs-up/down buttons, accepted vs. rejected code completions).

10.16 Building Preference Datasets for DPO and ORPO

The alignment of a model depends directly on the quality, nuance, and structural separation of its preference pairs.



1. The Anatomy of an Alignment Dataset

A preference dataset for DPO/ORPO requires three elements:

1. `prompt`: The grounding context, system prompt, and user query.
2. `chosen` (y_w): The demonstrably superior response (factually accurate, concise, safely bounded, structurally sound).
3. `rejected` (y_l): A subtly flawed response (contains hallucinations, unhandled edge cases, excessive verbosity, or violates safety guardrails).

2. Systematic Generation Strategies

- **Multi-Engine Sampling:** Generate completions for identical prompts across disparate model families (e.g., Claude 3.5 Sonnet, GPT-4o, LLaMA-3.1-70B, and smaller SFT checkpoints). Rank outputs based on objective criteria.
- **Targeted Negative Injection:** Take a pristine gold-standard completion (y_w) and generate programmatic variants (y_l) by:
  - Injecting factual contradictions or unverifiable claims.
  - Adding conversational filler and buzzwords.
  - Stripping required fields from an output JSON structure.
  - Reversing the logical flow of a reasoning step in a chain-of-thought trace.

3. The Margin Rule: Filtering Trivial and Ambiguous Pairs

Training on ambiguous preference pairs (where y_w is only marginally better than y_l) damages policy convergence.

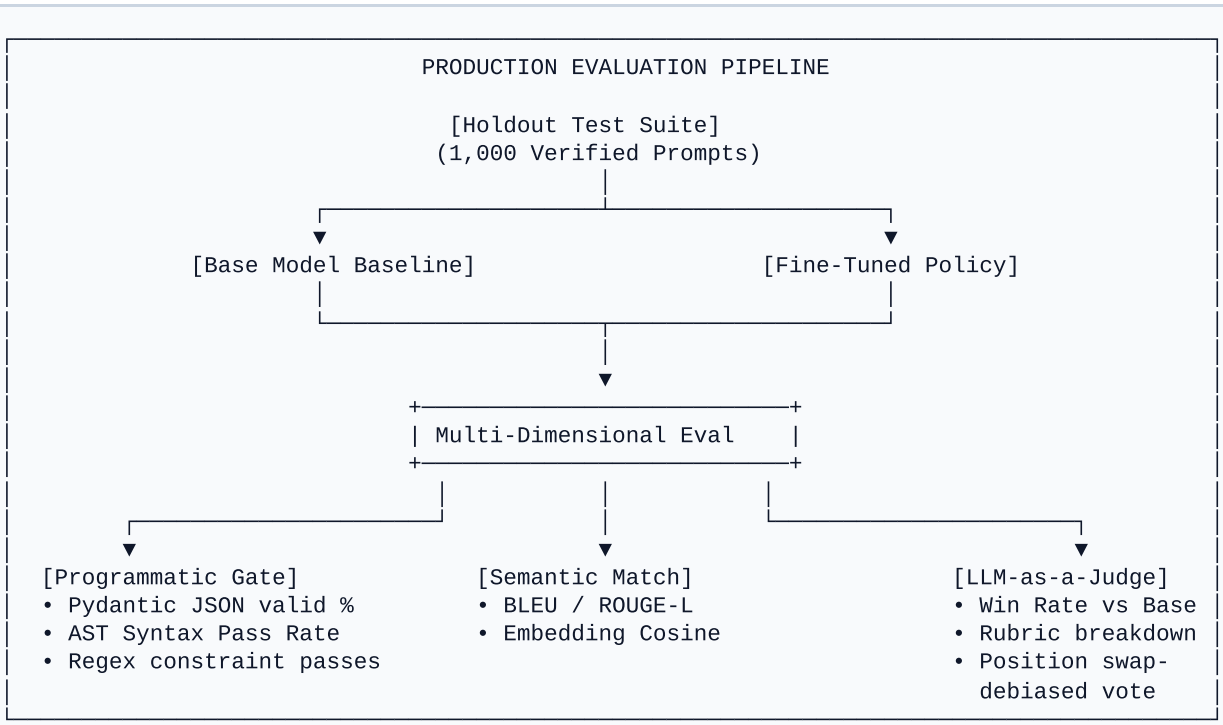
- Use an **LLM-as-a-Judge** scoring rubric evaluating candidate completions on a scale of 1 to 10:

  $$\Delta = \text{Score}(y_w) - \text{Score}(y_l)$$

- **Production Filter:** Retain only pairs where the quality margin exceeds an explicit threshold (e.g., $\Delta \geq 2.5$). Discard ties or marginal differences.
- **Length Bias Regularization:** Models optimize for length when given the chance. If y_w is consistently longer than y_l, the policy learns that verbosity equals quality. Ensure the token length distribution of y_w and y_l is balanced ($|y_w| \approx |y_l|$) across the dataset.

10.17 Evaluating a Fine-Tuned Model Against its Base Checkpoint

Evaluating task-specific models using general LLM leaderboards (e.g., MMLU, GSM8K, AlpacaEval) provides little insight into whether a model has mastered a targeted enterprise capability. Production evaluation requires **task-specific, test-set-isolated benchmarks**.



1. Deterministic & Programmatic Evaluation Gates

Before invoking expensive LLM judges, test outputs against deterministic criteria:

- **Schema Adherence Rate:** What percentage of outputs parse cleanly into the target schema without syntax errors?

$$\text{Parse Rate} = \frac{N_{\text{valid_json}}}{N_{\text{total}}} \times 100\%$$

- **Compilation / AST Execution Pass Rate:** For code generation models, run emitted code through language parsers or sandboxed unit tests (`pass@k`).
- **Regex Invariant Checks:** Ensure forbidden tokens, specific hallucinated patterns, or leaked delimiters never appear.

2. LLM-as-a-Judge with Position-Bias Mitigation

Using a frontier model (GPT-4o, Claude 3.5 Sonnet) as an evaluator requires controlling for **positional bias** (LLMs disproportionately prefer the candidate shown in Position A regardless of quality):

- **Symmetric Position Swapping:** For every test prompt x:
  - Evaluation 1: Present Candidate A as Option 1, Candidate B as Option 2.
  - Evaluation 2: Present Candidate B as Option 1, Candidate A as Option 2.
- A candidate is awarded a win if and only if it wins **both** orientations; otherwise, the evaluation is marked as a draw.

```
from pydantic import BaseModel, Field
from typing import Literal
from openai import OpenAI

client = OpenAI()

class JudgmentSchema(BaseModel):
    reasoning: str = Field(description="Step-by-step comparative critique against the
rubric.")
    winner: Literal["Model_A", "Model_B", "Tie"]

def evaluate_pairwise_debiased(prompt: str, response_base: str, response_tuned: str) -> str:
    """
    Evaluates fine-tuned model against base model using position-swap debiasing.
    """
    def run_judge(first: str, second: str):
        eval_prompt = f"""
        [USER QUERY]: {prompt}
        [RESPONSE 1]: {first}
        [RESPONSE 2]: {second}

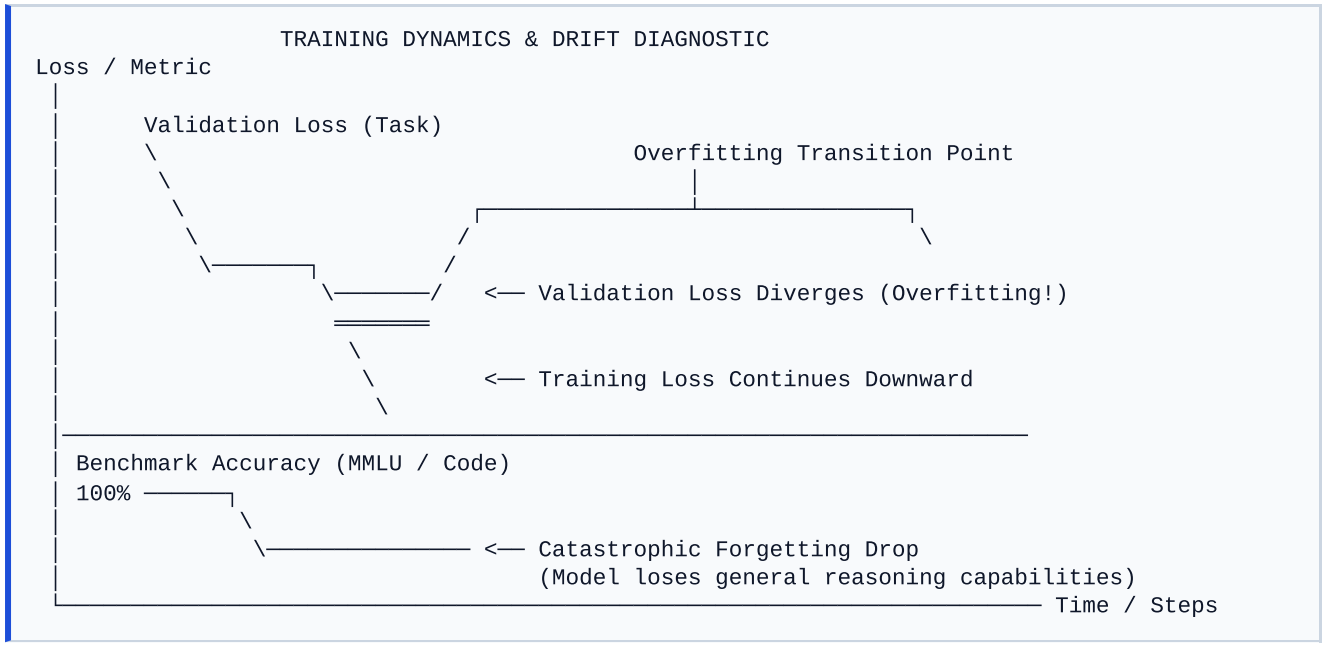
        Critique both responses strictly on factual correctness, tone, and conciseness.
        Decide which response is superior.
        """
        completion = client.beta.chat.completions.parse(
            model="gpt-4o",
            messages=[{"role": "system", "content": "You are an objective evaluation
judge."},
                      {"role": "user", "content": eval_prompt}],
            response_format=JudgmentSchema,
            temperature=0.0
        )
        return completion.choices[0].message.parsed.winner

    # Pass 1: Fine-tuned model in Position A
    win_pass1 = run_judge(response_tuned, response_base)
    # Pass 2: Fine-tuned model in Position B
    win_pass2 = run_judge(response_base, response_tuned)

    # Reconcile outcomes
    if win_pass1 == "Model_A" and win_pass2 == "Model_B":
        return "FINE_TUNED_MODEL_WINS"
    elif win_pass1 == "Model_B" and win_pass2 == "Model_A":
        return "BASE_MODEL_WINS"
    else:
        return "TIE_OR_POSITIONAL_BIAS"
```

10.18 Diagnosing Overfitting and Catastrophic Forgetting

Fine-tuning involves balancing specialized task learning against general language capabilities.



1. Indicators of Overfitting

- **Train vs. Validation Loss Divergence:** Training loss decreases monotonically while validation loss plateaus and begins curving upward.
- **Repetition & Output Degeneration:** The model begins repeating specific phrases, hardcoding dates, or reproducing training set examples verbatim.
- **Perplexity Spikes on Out-of-Domain Inputs:** The model exhibits high token perplexity when presented with standard English phrases that differ from the fine-tuning syntax.

2. Indicators of Catastrophic Forgetting

- **Anchor Benchmark Collapse:** A targeted drop in performance on foundational benchmarks (MMLU for broad knowledge, GSM8K for multi-step math, HumanEval for code) compared to the un-tuned base model.
- **Instruction Drift:** The model loses the ability to follow standard conversational instructions (e.g., failing to adhere to negative constraints like *"Do not include introductory pleasantries"*).

3. Automated Training Interventions

1. **Early Stopping:** Monitor validation loss every N evaluation steps. If validation loss fails to improve for 3 consecutive checks (patience = 3), halt training and restore the best checkpoint.
2. **Replay Buffer Regularization:** Mix **10% to 20% general instruction data** (e.g., ShareGPT, OpenOrca, or base pre-training tokens) into the domain dataset. This provides an active gradient signal that protects foundational attention patterns.
3. **Weight Decay & Dropout:** Set `weight_decay = 0.01` to `0.1` in the AdamW optimizer and introduce adapter dropout (`lora_dropout = 0.05` to `0.1`) to penalize co-adaptation of weights.

10.19 Cost-Benefit Analysis: Fine-Tuning vs. Strong RAG + Prompting

Before committing engineering resources to a production fine-tuning pipeline, conduct an objective cost-benefit audit comparing it against a well-engineered RAG baseline.

1. The Cost Equation Framework

> $$\text{Cost}_{\text{Total}} = \text{Cost}_{\text{Upfront}} + (\ \text{Cost}_{\text{Inference}} \times \text{Volume}\ ) + \text{Cost}_{\text{Maintenance}}$$

Scenario: 1,000,000 Production Invocations per Month

```
+-------------------------------------------------------------------------------------+
| WORKLOAD ASSUMPTIONS:                                                               |
| • Prompt Size (Few-Shot Baseline):  2,500 input tokens + 300 output tokens          |
| • Prompt Size (Fine-Tuned Model):     300 input tokens + 300 output tokens          |
| • Model Baseline: GPT-4o ($2.50 / 1M in, $10.00 / 1M out)                           |
| • Fine-Tuned Model: LLaMA-3-8B self-hosted on 1x A10G ($1.00 / hr Cloud Instance)   |
+-------------------------------------------------------------------------------------+
```

Option A: Few-Shot Prompting with Frontier API (GPT-4o)

- Input Cost: *1,000,000* × *2,500* tokens = *2.5*B tokens × *$2.50 / 1*M = ***$6,250 / month***
- Output Cost: *1,000,000* × *300* tokens = *300*M tokens × *$10.00 / 1*M = ***$3,000 / month***
- Total Monthly Cost: **$9,250 / month** ($0 upfront training cost).

Option B: Fine-Tuned 8B Model Hosted on Cloud GPU

- Upfront Training: 8 hours on 1x A100 GPU ≈ ***$35.00*** (one-time compute) + dataset curation.
- Hosting Infrastructure: 1x AWS `g5.xlarge` (A10G GPU) running 24/7 = *730* hours × *$1.01* / hr = ***$737.30 / month***.
- Token Cost: **$0** (Unlimited tokens within hardware throughput limits).
- Total Monthly Cost: **$737.30 / month**.
- **Net Financial Savings:** Over **$8,500 / month (a 92% cost reduction)**, while achieving lower p99 latency by cutting input token prefill length by 88%.

10.20 Production Fine-Tuning Tooling: Axolotl, Unsloth, and LLaMA-Factory

Modern open-source frameworks streamline the low-level PyTorch/CUDA boilerplate needed to train LLMs.

```
┌──────────────────────────────────┬──────────────────────────────────┬──────────────────────────────────┐
│            AXOLOTL               │            UNSLOTH              │          LLAMA-FACTORY           │
├──────────────────────────────────┼──────────────────────────────────┼──────────────────────────────────┤
│ • Production & Cluster Standard  │ • Single-GPU Speed Optimizer     │ • Low-Code Enterprise Suite      │
│ • Declarative YAML config files  │ • Custom hand-written OpenAI     │ • Web UI dashboard + CLI         │
│ • Deep integration with FSDP,    │   Triton CUDA kernels            │ • Supports 100+ model families   │
│   DeepSpeed, FlashAttention-2    │ • 2x–5x training speedup         │ • Automated SFT, DPO, PPO,       │
│ • Reproducible enterprise runs   │ • Slashes VRAM by 70–80%         │   and KTO workflows              │
└──────────────────────────────────┴──────────────────────────────────┴──────────────────────────────────┘
```

1. Axolotl (The Production GitOps Standard)

Axolotl standardizes training through declarative YAML configuration files, isolating pipeline code from hyperparameters and dataset schemas:

```yaml
# axolotl_config.yaml
base_model: meta-llama/Meta-Llama-3-8B-Instruct
model_type: LlamaForCausalLM
tokenizer_type: AutoTokenizer

load_in_4bit: true
adapter: qlora
lora_r: 16
lora_alpha: 32
lora_dropout: 0.05
lora_target_modules:
  - q_proj
  - k_proj
  - v_proj
  - o_proj
  - gate_proj
  - up_proj
  - down_proj

datasets:
  - path: ./enterprise_training_data.jsonl
    type: sharegpt
    conversation: llama3

dataset_prepared_path: ./last_run_prepared
val_set_size: 0.05
output_dir: ./lora_enterprise_checkpoint

sequence_len: 4096
sample_packing: true      # Packs multiple sequences into one context window (3x speedup)
flash_attention: true

learning_rate: 0.0002
optimizer: paged_adamw_8bit
lr_scheduler: cosine
gradient_accumulation_steps: 4
micro_batch_size: 2
num_epochs: 3
```

Execution is a single command:

```
accelerate launch -m axolotl.cli.train axolotl_config.yaml
```

2. Unsloth (Ultra-Fast Single-GPU QLoRA)

Unsloth rewrites core PyTorch attention, MLP, RoPE, and cross-entropy backward passes directly in **OpenAI Triton**:

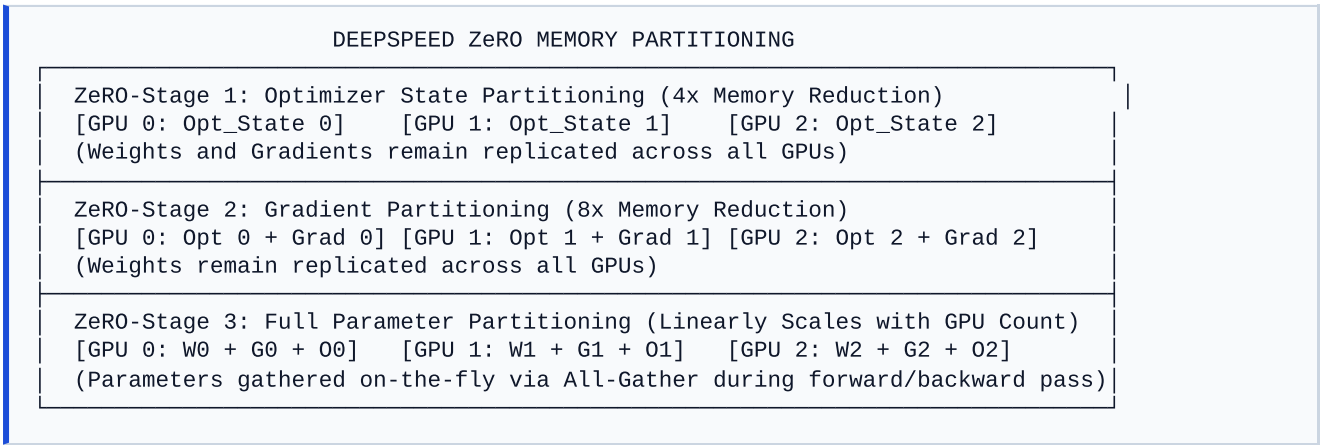
- Replaces standard PyTorch autograd compute graphs with manually derived, fused mathematical kernels.
- **Performance:** Provides **2x to 5x faster training speeds** and **slashes VRAM consumption by 70% to 80%**, allowing fine-tuning of 8B models on 8 GB VRAM GPUs and 70B models on a single 80 GB GPU.

3. LLaMA-Factory (Unified Low-Code UI & Workflow Engine)

- Provides a Gradio-based web interface (`llamafactory-cli webui`) for configuring datasets, LoRA targets, prompt templates, and evaluation splits without writing code.
- Provides a unified codebase supporting Supervised Fine-Tuning, Reward Modeling, PPO, DPO, ORPO, and KTO across dozens of model architectures.

10.21 Distributed Fine-Tuning: DeepSpeed ZeRO Stages, FSDP, and Scaling

When a model's memory footprint (weights + gradients + optimizer states + activations) exceeds the physical VRAM capacity of a single GPU, training must be distributed across multi-GPU or multi-node clusters.



1. DeepSpeed ZeRO (Zero Redundancy Optimizer) Stages

Standard Data Parallelism (DDP) replicates the entire model, gradients, and optimizer states across every GPU, wasting memory on identical parameters. **DeepSpeed ZeRO** eliminates this redundancy by sharding these states across the cluster:

ZeRO-Stage 1 (Optimizer State Sharding)

- The AdamW optimizer states (12 bytes per parameter) are divided equally across all N_{GPUs} devices.
- Every GPU holds only $1 / N_{GPUs}$ of the optimizer states.
- Communication Overhead: **0% increase** compared to standard DDP.
- Memory Reduction: **~4x memory reduction**, allowing larger batch sizes.

ZeRO-Stage 2 (Gradient + Optimizer Sharding)

- Both the optimizer states and the gradients are sharded across the cluster.
- Each GPU retains only the gradients corresponding to its assigned slice of the optimizer states.
- Communication Overhead: **0% increase** (uses standard Reduce-Scatter and All-Gather collectives).
- Memory Reduction: **~8x memory reduction**.

ZeRO-Stage 3 (Full Parameter Sharding — FSDP Equivalence)

- **Model weights, gradients, and optimizer states are all sharded across all GPUs.**
- A GPU holds only a small fraction ($1 / N_{GPUs}$) of the model weights in static memory.
- **Dynamic Ring Communication:**
  - *Forward Pass:* When Layer L executes, the cluster initiates an `All-Gather` collective to fetch Layer L's weights from the holding GPU, computes the activations, and immediately purges Layer L's weights from memory.
  - *Backward Pass:* Weights are gathered again via `All-Gather` to calculate gradients, followed by a `Reduce-Scatter` collective to distribute gradient updates back to their owning GPUs.
- **Trade-Off:** Unlocks training of massive models (e.g., 70B–405B) across commodity GPU clusters at the cost of **~50% higher inter-GPU communication traffic**.

2. PyTorch Fully Sharded Data Parallel (FSDP)

FSDP is PyTorch's native implementation of ZeRO-Stage 3. Unlike DeepSpeed (which requires third-party C++ libraries and distinct runtime wrappers), FSDP is built into core PyTorch (`torch.distributed.fsdp`):

- Wraps individual Transformer layers or blocks (`auto_wrap_policy`).
- Supports native mixed precision (BF16 compute with FP32 master weight accumulation).
- Enables **CPU Offloading**, swapping sharded model weights and optimizer states to host RAM over PCIe when cluster GPU memory is saturated.

10.22 Model Merging: SLERP, TIES, DARE, and MergeKit

Model Merging combines the weights of multiple specialized fine-tuned models derived from the same base checkpoint **without running further gradient training or spending additional compute**.

```
┌──────────────────────────────────────┬──────────────────────────────────────┬──────────────────────────────────────┐
│        LINEAR WEIGHT AVG             │    SPHERICAL LINEAR (SLERP)          │        TIES / DARE MERGING           │
├──────────────────────────────────────┼──────────────────────────────────────┼──────────────────────────────────────┤
│ W_merge = 0.5·W_A + 0.5·W_B          │ Interpolates along spherical         │ 1. Trims small parameter deltas      │
│                                      │ manifold arc.                        │ 2. Resolves sign interference        │
│ Destroys geometric magnitude;        │                                      │ 3. Elects disjoint weights           │
│ collapses model capabilities.        │ Preserves vector scale & angle.      │ Combines 5+ specialized models.      │
└──────────────────────────────────────┴──────────────────────────────────────┴──────────────────────────────────────┘
```

1. Naive Linear Averaging and Why It Fails

Simply averaging weights ($W_{new} = (W_A + W_B) / 2$) degrades performance:

- High-dimensional neural network weights do not sit in flat Euclidean space; they reside on a complex curved manifold.
- Averaging vectors cuts straight through the interior of the hypersphere, reducing the norm/magnitude of weight vectors ($\|W_{avg}\| \ll \|W_A\|$) and corrupting downstream activation distributions.

2. Spherical Linear Interpolation (SLERP)

SLERP interpolates between two weight vectors along the geometric arc of a high-dimensional hypersphere, keeping vector magnitude and angular relationships intact:

$$\text{SLERP}(W_1, W_2 ; t) = \frac{\sin((1-t)\theta)}{\sin(\theta)} W_1 + \frac{\sin(t\theta)}{\sin(\theta)} W_2$$

where:

- $\theta = \arccos((W_1 \cdot W_2) / (\|W_1\| \|W_2\|))$ is the angle between the two weight tensors.
- $t \in [0.0, 1.0]$ is the interpolation parameter.
- **Application:** Ideal for blending two models (e.g., merging a high-reasoning math model with an instruction-following chat model).

3. TIES-Merging (Trimming, Electing Sign, and Disjoint Merging)

Introduced by Yadav et al. (NeurIPS 2023), **TIES-Merging** resolves the **Parameter Interference Problem** when combining three or more models:

1. **Trim:** Keep only the top $k\%$ most significant parameter updates relative to base model W_o ($|\Delta W| = |W_i - W_o|$); set the lowest $(100 - k)\%$ small-magnitude values to zero (reducing noise).
2. **Elect Sign:** Weight updates across different models often conflict in direction (+ vs. −). TIES resolves this by calculating the majority sign vector across all models:

$$\gamma = \text{sgn}\left(\sum_{m=1}^{M} \text{Trimmed}(\Delta W_m) \right)$$

3. **Disjoint Merge:** Average only those updates that agree with the majority sign vector γ, discarding conflicting directions.

4. DARE (Drop And REscale)

Introduced by Yu et al. (2024), **DARE** uses a dropout-style sparsification step:

1. Randomly drops parameter deltas with probability p (often $p = 0.9$, pruning 90% of updates).
2. Rescales remaining parameters by $1 / (1 - p)$ to preserve the expectation of the weight update tensor:

$$\Delta W_{\text{DARE}} = \frac{1}{1-p} \cdot \text{Bernoulli}(1-p) \odot \Delta W$$

3. DARE can be combined with TIES or SLERP, enabling the combination of many diverse task-specific models into a single base checkpoint.

5. Production Merge Implementation with MergeKit

MergeKit is the industry standard open-source framework for declarative model merging. It executes out-of-core merging using CPU host RAM, allowing models to be merged without requiring high-end GPUs.

```
# merge_config.yaml
merge_method: ties
base_model: meta-llama/Meta-Llama-3-8B
models:
  - model: meta-llama/Meta-Llama-3-8B-Instruct
    parameters:
      density: 0.7
      weight: 0.4
  - model: my-org/llama-3-8b-python-coder
    parameters:
      density: 0.5
      weight: 0.3
  - model: my-org/llama-3-8b-legal-reasoner
    parameters:
      density: 0.5
      weight: 0.3
parameters:
  normalize: true
  int8_mask: true
dtype: bfloat16
```

Execute the merge via CLI:

```
mergekit-yaml merge_config.yaml ./merged_production_model --cuda --allow-crimes
```

Technical Summary of Module 10 Part 3

Section	Algorithmic / Core Mechanism	Mathematical Formulation / Invariant	Primary Failure Mode Addressed
10.15 ORPO & KTO	Single-stage alignment; Prospect theory loss aversion	$L_{ORPO} = L_{SFT} + \lambda L_{OR}$; Unpaired binary KTO updates	Eliminates reference model memory bloat and separate multi-stage SFT pipelines
10.16 Preference Data	LLM-as-a-judge filtering; Margin enforcement	$\Delta = \text{Score}(y_w) - \text{Score}(y_l) \geq \tau$; Token length parity	Policy collapse from noisy pairs and model learning length bias
10.17 Evaluation	Programmatic schema gates + Debiased LLM judge	Symmetric position swapping ($A/B \rightarrow B/A$ swap tests)	Subjective evaluation errors and model positional bias
10.18 Drift Diagnosis	Early stopping & Experience replay buffers	10%–20% general instruction data blended in domain mix	Divergence of validation loss and catastrophic forgetting
10.19 Cost-Benefit	Total Cost of Ownership (TCO) modeling	$C_{Total} = C_{Fixed} + (C_{Token} \times \text{Volume})$	Over-investing in fine-tuning for low-volume or dynamic-knowledge applications
10.20 Tooling	Axolotl (YAML), Unsloth (Triton), LLaMA-Factory (UI)	Triton fused kernels provide 2x–5x speedup; Sample packing	High barrier to entry and slow execution speeds in raw PyTorch
10.21 Distributed Scaling	DeepSpeed ZeRO 1/2/3 and PyTorch FSDP	Stage 1 (Optimizer) → Stage 2 (+ Gradients) → Stage 3 (+ Parameters)	CUDA Out-of-Memory crashes when model size exceeds single-GPU VRAM
10.22 Model Merging	Hyperspherical interpolation and interference resolution	$\text{SLERP}(W_1, W_2; t)$; TIES Trim-Elect-Merge; DARE drop-rescale	Destructive parameter interference when naively averaging model weights